

FIRST.PY

```
import matplotlib
print("Matplotlib version: ", matplotlib.__version__)
print("Yayyyy!!!! :] <3")
```

LINE PLOT.PY

```
import matplotlib.pyplot as plt
# plt.figure(1) #create new figure
# plt.plot([1,2,3,4]) #matplotlib will consider this seq as Y sequence
# plt.title("Line plot (only 1 sequence)")
# plt.xlabel("X Axis")
# plt.ylabel("Y Axis")
#
# x = [2,3,4,5]
# y = [9,4,7,10]
# plt.figure(2) #create new figure
# plt.plot(x,y, label = "Line A")
# plt.legend()
# plt.title("Line plot (2 sequences)")
# plt.xlabel("X Axis")
# plt.ylabel("Y Axis")

plt.figure("Faru")

x1 = [0.7,2,3,4]
y1 = [4,5,6,9.5]
x2 = [0.5,2,3,4.3,5]
y2 = [5,3,7,8,9]

plt.plot(x1,y1,label="Line A",marker='^') #label for line (in legend)
plt.plot(x2,y2,label = "Line B",marker='o')

plt.legend() #show labels as legend

plt.title("Line plot with some extra styling ;)")
```

```

plt.xlabel("X Axis")
plt.ylabel("Y Axis")

plt.grid(True, linestyle = '--') #grid in plot area

plt.xticks([0,1,2,3,4,5,6], [ "", "One", "Two", "Three", "Four", "Five", "Six"])
plt.yticks([3,4,5,6,7,8,9,10]) #custom ticks

plt.text(1.5,6.5, ". Hiii <3", color = "red") #add text at a specific pos on plot

plt.annotate("Annotation :0", (3.9,9.0), (4.5,9.5), arrowprops=dict(facecolor='black'))

# plt.savefig('Line_plot.png', dpi=300)
# plt.savefig('Line_plot.pdf')
# plt.savefig('Line_plot.jpg')

plt.show()

# plt.figure() used -> to show both plots on different windows
# (as different figures)

```

PRACTICEE.PY

```

import matplotlib.pyplot as plt
import numpy as np

x = np.array([1,2,3,4])
y = x ** 2

plt.figure("Line Plot Example", figsize=(10,6))
plt.plot(x,y,label='Line A',marker='^')
plt.legend()
plt.xlabel('X Axis')
plt.ylabel('Y Axis')
plt.xticks([0,0.5,1,1.5,2,2.5,3,3.5,4,4.5,5], [ "", "", "One", "", "Two", "", "Three", "", "Four", "", "Five"])

for i, (xi,yi) in enumerate(zip(x,y)):

```

```
plt.annotate(f"({xi},{yi})", (xi,yi),(xi,yi+0.5))
```

```
plt.show()
```

BAR PLOT.PY

```
import matplotlib.pyplot as plt
```

```
plt.bar(['A', 'B', 'C', 'D'],[1,2,3,4],label="Some label")  
plt.legend()  
plt.title("Bar Chart")  
plt.xlabel("X Axis")  
plt.ylabel("Y Axis")  
plt.grid(axis="y") #only y axis grids  
plt.show()
```

SCATTER PLOT.PY

```
import matplotlib.pyplot as plt
```

```
plt.scatter([1,2,3,4],[3,4,7,9],label="Some label")  
plt.legend()  
plt.title("Scatter Plot")  
plt.xlabel("X Axis")  
plt.ylabel("Y Axis")  
plt.grid(axis="x") #only y axis grids  
plt.show()
```

HISTOGRAM.PY

```
import matplotlib.pyplot as plt
```

```
plt.figure("Histogram :D", figsize=(10,5)) #10 inches wide - ,5 inches tall |  
plt.hist([1,23.63,45,65,43],5,label="Some label")  
plt.legend()  
plt.grid(True)  
plt.show()
```

SQUARE.PY

```
import matplotlib.pyplot as plt

x = [1,1,2,2,1]
y = [1,2,2,1,1]

plt.figure(figsize=(6,6))
plt.plot(x,y,label='Square',color='black')
plt.legend()
plt.gca().set_aspect('equal', adjustable='box')
plt.title('Square Shape')
plt.grid(True)
plt.xlim(0,6)
plt.ylim(0,6)

plt.show()
```

INTERACTIVE SINE WAVE.PY

```
# work pending
# sync check box & wave visibility after reset
# keep colors of both waves diff

import matplotlib.pyplot as plt
from matplotlib.widgets import Slider, Button, CheckButtons, RadioButtons, TextBox
import numpy as np

updating_reset = False

x = np.linspace(0,2*np.pi,400) #400 points from 0 to 2*pi
initial_freq = 1 #starting frequency
initial_amp = 1 #starting amplitude

y_sin = initial_amp * np.sin(initial_freq * x) #initial sine wave data
y_cos = initial_amp * np.cos(initial_freq * x) #initial cosine wave data
```

```

#Create figure & axes
fig, ax = plt.subplots(figsize=(12,6)) #creates plotting area
#fig for entire window
#ax for plot area inside figure
plt.subplots_adjust(left=0.2,bottom=0.35) #leaves space at bottom for sliders

[line_sin] = ax.plot(x,y_sin,visible=True,lw=2,label='Sine') #line obj needed to update data
dynamically
[line_cos] = ax.plot(x,y_cos,visible=True,lw=2,label='Cosine')
#lw == linewidth - thickness of plotted line

ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title("Interactive Sine Wave")

#Slider
# axes >> [left, bottom, width, height]
# slider >> (slider axis, label, min_val, max_val, initial slider value)

ax_freq = plt.axes([0.1,0.15,0.8,0.03])
freq_slider = Slider(ax_freq, 'Frequency', 0.1, 5.0, valinit=initial_freq)

ax_amp = plt.axes([0.1,0.1,0.8,0.03])
amp_slider = Slider(ax_amp, 'Amplitude', 0.1, 5.0, valinit=initial_amp)

#Checkbox for sin/cos
check_ax = plt.axes([0.02,0.7,0.1,0.15]) #position of checkboxes
check = CheckButtons(check_ax,['Sine','Cosine'],[True,True])

#Radio buttons for color
color_ax = plt.axes([0.02,0.5,0.1,0.15])
radio = RadioButtons(color_ax,('blue', 'red', 'green', 'orange'))

#Textbox
ax_box = plt.axes([0.15,0.01,0.4,0.05])
text_box = TextBox(ax_box, 'Plot Title: ', initial='Interactive Sine & Cosine Waves')

```

```

#Update function
def update(val):
    #current slider value
    amp = amp_slider.val
    freq = freq_slider.val

    line_sin.set_ydata(amp * np.sin(freq * x)) # .set_ydata() >> updates y-value of that line
    line_cos.set_ydata(amp * np.cos(freq * x))
    #x value remains same, only y value changes

    fig.canvas.draw_idle()
    # fig.canvas >> drawing area of figure
    # .draw_idle() >> tells Matplotlib > Redraw the figure when idle, don't redraw immediately for
    every tiny slider move

freq_slider.on_changed(update)
amp_slider.on_changed(update)

def toggle_lines(label):
    if label == 'Sine':
        line_sin.set_visible(not line_sin.get_visible()) #get_visible >> returns bool, if line visible >>
        True & vice versa
        #not flips it
    elif label == 'Cosine':
        line_cos.set_visible(not line_cos.get_visible())
    fig.canvas.draw_idle()

check.on_clicked(toggle_lines)

def change_color(label):
    if line_sin.get_visible():
        line_sin.set_color(label)
    if line_cos.get_visible():
        line_cos.set_color(label)
    fig.canvas.draw_idle()

radio.on_clicked(change_color)
#Reset Button

```

```

button_ax = plt.axes([0.8,0.025,0.1,0.04])
button = Button(button_ax,'Reset')

def reset(event):
    global updating_reset
    updating_reset = True #temporarily disable callback

    freq_slider.reset()
    amp_slider.reset()

    line_sin.set_visible(True) #set visibility
    line_cos.set_visible(True)

    line_sin.set_color('blue') #set color
    line_cos.set_color('orange')

    default_title = 'Interactive Sine & Cosine Waves'
    ax.set_title(default_title)
    text_box.set_val(default_title)

    #check checkboxes
    # check.set_active(0) if not check.get_status()[0] else None # <expression1> if <condition>
else <expression2>
    # check.set_active(1) if not check.get_status()[1] else None

    fig.canvas.draw_idle()

button.on_clicked(reset)

def submit_title(text):
    ax.set_title(text) #update plot title
    fig.canvas.draw_idle()

text_box.on_submit(submit_title)

# plt.savefig('Sine_Wave.png', dpi=300)
plt.show()

```